



MCS5210 – Control Engineering IV

Dr. Ado Haruna

Dept. of Mechatronics Engineering

Bayero University, Kano

aharuna.mct@buk.edu.ng



Discrete PID Controller

- The continuous PID controller can be written as

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \int_0^T e(t) dt + T_d \frac{de(t)}{dt} \right]$$

where T_d and T_i are the derivative and integral times respectively.

- The derivative and integral terms can be expressed discretely as

$$\frac{de(t)}{dt} = \frac{e_k - e_{k-1}}{T} \quad \text{and} \quad I_k = I_{k-1} + T e_k$$

Thus

$$u_k = K_p \left[e_k + \frac{1}{T_i} (I_{k-1} + T e_k) + T_d \frac{e_k - e_{k-1}}{T} \right]$$



Positional PID



Discrete PID Controller: Velocity PID

But

$$u_{k-1} = K_p \left[e_{k-1} + \frac{1}{T_i} (I_{k-2} + T e_{k-1}) + T_d \frac{e_{k-1} - e_{k-2}}{T} \right]$$

$$\begin{aligned} u_k - u_{k-1} &= \Delta u_k \\ &= K_p \left[e_k - e_{k-1} + \frac{1}{T_i} (I_{k-1} - I_{k-2} + T e_k - T e_{k-1}) \right. \\ &\quad \left. + T_d \frac{e_k - e_{k-1} - e_{k-1} + e_{k-2}}{T} \right] \end{aligned}$$

$$\Delta u_k = K_p \left[e_k - e_{k-1} + \frac{1}{T_i} (T e_k) + T_d \frac{e_k - 2e_{k-1} + e_{k-2}}{T} \right]$$

$$u_k = u_{k-1} + \Delta u_k$$



Velocity PID

The **velocity form** PID may also be expressed as in terms of the error as

$$\Delta u_k = (K_p + K_i + K_d) e_k - (K_p + 2K_d) e_{k-1} + K_d e_{k-2}$$

$$\text{where } K_i = \frac{K_p T}{T_i} \text{ and } K_d = \frac{K_p T_d}{T}$$



Computer PID Implementation

- When a digital computer is used to implement a control law, the ideal sequence of operation is the following:
 1. Wait for clock interrupt
 2. Read analog input
 3. Compute control signal
 4. Set analog output
 5. Update controller variables
 6. Goto to 1
- The main drawback in computer based control is that input data to the processor must be **sampled** via an A/D converter.
- Thus, only snap shots of the system data is available which makes the **sampling frequency** very important.
- The rule of thumb is to sample a signal at about **ten times** higher than the highest frequency present in the signal.
- Another important thing to consider in integral controllers is that an **anti-windup** mechanism must be implemented.
- Anti-windup can easily be implemented in the velocity form PID setting appropriate upper and lower **limits** for the control signal.



Integral Windup

- Integral action is employed in feedback controllers to **eliminate** steady state error.
- In implementing integral controllers it is necessary to prevent integral **windup**.
- This is a phenomenon that is caused by interaction of **integral actions** and **saturations**.
- This is because all actuators have **limitations**.
- For instance, a valve cannot be opened more than 100% and a motor cannot exceed its maximum speed limit.
- Windup causes the feedback loop to be **broken** resulting in large transients, oscillation and even instability.
- Anti windup can easily be implemented in the velocity PID by **limiting** the control signal within the **normal** operating region of the actuator.



Computer PID Implementation

Example: Consider the model of a tank system given by the transfer function

$$G(s) = \frac{15}{30s + 1}$$

- i. Create a discrete model of the tank system.
- ii. Implement a discrete velocity form PID controller for the system so that the level is maintained as 10 cm above the orifice.
- iii. Plot the reference, output and reference signals.
- iv. Tune the PID controller to achieve the best results.



MALAB Example: Code

```
t = 0:0.1:200;
u = zeros(length(t),1);
y = zeros(length(t),1);
r = zeros(length(t),1);
yk = 0;
uk = 0;
ek = 0;
rk = 30;
kp = 1;
ki = 1;
kd = 0;
ek_2 = 0;
ek_1 = 0;
ek = 0;
delta_uk = 0;
for k = 1:1:length(t)
    % system model
    yk = .997*yk + 0.0499*uk;
    %uk = 1 %unit step open loop
```



MALAB Example: Code

```
%PID controller
    ek_2 = ek_1;
    ek_1 = ek;
    ek = rk - yk;
    delta_uk = (kp+ki+kd)*ek - (kp+2*kd)*ek_1 + kd*ek_2;
    uk = uk + delta_uk;
    %anti windup
    if (uk < 0)
        uk = 0;
    elseif (uk > 10);
        uk = 10;
    end
    %store results
    y(k) = yk;
    u(k) = uk;
    r(k) = rk;
end
plot(t,r, 'b', 'linewidth', 2)
hold on
plot(t,y, '--y', 'linewidth', 2)
plot(t,u, 'r')
```